

Testando modelos pyruleanalyzer em Arduino e ESP32 reais

Guia do script `examples/arduino_hardware_test.py` — do treino em Python até a validação na placa física

1. O que este script faz

O **pyruleanalyzer** já exporta um modelo treinado (Decision Tree, Random Forest ou GBDT) para um sketch Arduino autocontido, com as árvores convertidas em arrays `const` e uma função `pyra_predict()` em C. O que faltava era responder à pergunta seguinte: **a placa realmente classifica igual ao Python?**

O script `examples/arduino_hardware_test.py` responde isso com um teste *hardware-in-the-loop* (HIL): ele treina o modelo, gera um sketch de teste, compila, grava na placa, envia amostras reais pela porta serial e compara, uma a uma, a classe devolvida pelo microcontrolador com a classe prevista em Python — medindo de quebra a latência de cada inferência e o uso real de Flash e SRAM.

O sketch de **produção** lê sensores dentro de `read_features()`; por isso ele não pode ser testado sozinho — os dados vêm do mundo físico. O truque do teste é inverter o fluxo: o PC passa a ser a fonte dos dados, e o sketch de **teste** reaproveita exatamente os mesmos arrays e a mesma `pyra_predict()`, trocando apenas a leitura dos sensores por um protocolo serial. Valida-se, portanto, o mesmo código que vai para o campo.

Como o teste funciona: o PC vira a fonte dos dados

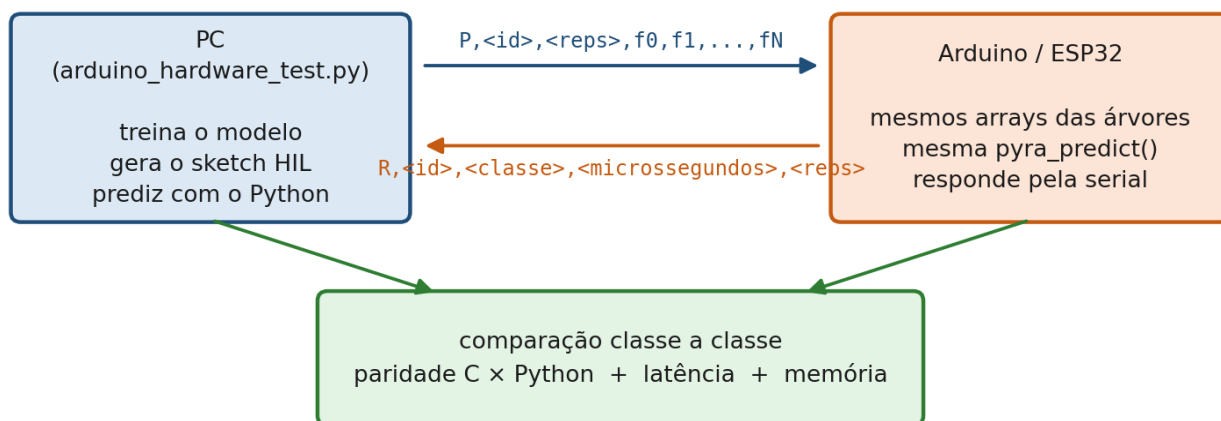


Figura 1 — o PC envia o vetor de features; a placa devolve a classe e o tempo de inferência. O que sobra é comparar.

2. Antes de começar: pré-requisitos

Instale uma vez só, na máquina que vai rodar o teste:

```
pip install pyserial # comunicação serial com a placa

# arduino-cli (compilar e gravar) - https://arduino.github.io/arduino-cli/
winget install ArduinoSA.CLI # Windows
arduino-cli core update-index
arduino-cli core install arduino:avr # Uno / Nano / Mega / Leonardo
arduino-cli core install esp32:esp32 # ESP32 (exige a URL do board manager)
```

```
arduino-cli board list
```

```
# confirme que a placa aparece
```

- **Windows:** clones de Nano e ESP32 precisam do driver CH340 ou CP2102.
- **g++** (opcional, mas recomendado): habilita a etapa 3, que valida o C no PC sem placa nenhuma.
- **Feche o Monitor Serial** da Arduino IDE antes de rodar: a porta é exclusiva.

3. O pipeline de teste, etapa por etapa

As 7 etapas do pipeline

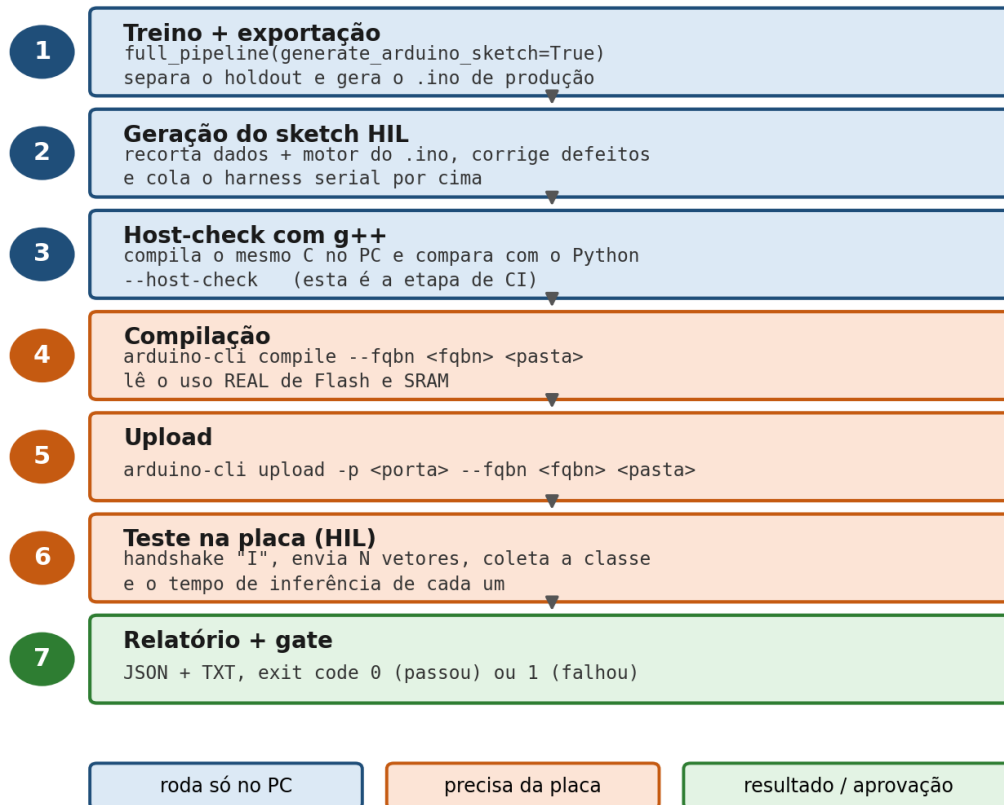


Figura 2 — as etapas em azul rodam só no PC; as laranjas exigem a placa conectada.

Etapa 1 — Treino e exportação

O script carrega o dataset (embutido do scikit-learn ou o seu CSV), separa um conjunto de teste que **não** participa do treino e chama `full_pipeline(generate_arduino_sketch=True)`. Saem daí o .ino de produção e a estimativa de memória (`memory_check`).

Etapa 2 — Geração do sketch HIL

O script recorta do .ino tudo o que vem antes do marcador `ARDUINO SKETCH SECTION` — ou seja, os dados das árvores e o motor de inferência — e cola por cima o *harness* serial. O resultado fica em `files/hil/<nome>_hil/<nome>_hil.ino` (o Arduino exige que a pasta tenha o mesmo nome do arquivo).

De quebra, esta etapa conserta dois defeitos do gerador: (1) o bloco `Feature order`: sai sem `//` e não compila; (2) o sketch de produção declara `float features[]` e chama `pyra_predict((const double*)features)` — um cast inválido que lê lixo em qualquer plataforma onde `float != double`. O harness declara o buffer como `double` e não usa cast.

Etapa 3 — Host-check com g++ (sem hardware)

Antes de gastar tempo gravando, o mesmo código C é compilado no PC com g++ e alimentado com os vetores de teste. Se a paridade C × Python já falha aqui, o problema está no modelo ou no exportador — e não na placa. É esta a etapa que você roda em integração contínua.

Etapa 4 — Compilação para a placa

O `arduino-cli compile` devolve o uso **real** de memória; o script extrai as linhas `Sketch uses ... bytes` e `Global variables use ... bytes` e as coloca no relatório ao lado da estimativa. O número real ser maior que o estimado é normal — entra o runtime do Arduino. O que importa é a porcentagem final.

Etapa 5 — Upload

Gravação via `arduino-cli upload -p <porta>`. Se a placa já estiver com o sketch HIL, use `--skip-upload`; para pular também a compilação, `--skip-compile`.

Etapa 6 — O teste na placa

O script abre a serial (abrir a porta reseta placas AVR, por isso ele espera `--boot-delay` segundos), faz o handshake com o comando `I` e então envia as amostras. Cada resposta traz a classe e o tempo total de `--reps` inferências, para que a resolução de micros() não domine a medida.

Etapa 7 — Relatório e critério de aprovação

Sai um `.json` e um `.txt` em `--report-dir`. O processo termina com **exit code 0** (passou) ou **1** (falhou), pronto para servir de gate em CI. Reprova se: a paridade ficar abaixo de `--min-parity` (padrão 100%), houver qualquer timeout ou resposta inválida, o host-check falhar, ou Flash/SRAM estourarem os limites da placa.

4. Como executar

4.1 Sem placa nenhuma (comece por aqui)

Valida o pipeline inteiro até a etapa 3. É o comando ideal para a primeira vez e para rodar em CI:

```
python examples/arduino_hardware_test.py --dataset iris \
  --host-check --no-hardware
```

4.2 Ciclo completo em uma placa real

```
# Arduino Uno na COM5 (Linux/Mac: /dev/ttyUSB0 ou /dev/ttyACM0)
python examples/arduino_hardware_test.py --dataset iris \
  --board uno --port COM5 --host-check
```

Omitindo `--port`, o script tenta descobrir a placa sozinho com `arduino-cli board list`.

4.3 Outros cenários úteis

```
# ESP32 com Random Forest, 200 amostras e 50 repetições por amostra
python examples/arduino_hardware_test.py --dataset wine \
  --model "Random Forest" --n-estimators 10 --max-depth 6 \
  --board esp32 --port COM7 --n-samples 200 --reps 50

# Seu próprio CSV, com a placa já gravada
python examples/arduino_hardware_test.py --csv files/dataset_iris.csv \
  --target Target --board nano --port COM3 --skip-upload

# Reaproveitando um modelo já treinado
```

```
python examples/arduino_hardware_test.py --model-pkl files/final_model.pkl \
--board mega --port COM4
```

4.4 Principais opções

Opção	Para que serve
--dataset / --csv	Dataset embutido (iris, wine, breast_cancer, digits) ou o seu CSV, com --target definindo a coluna alvo.
--model	"Decision Tree" (padrão), "Random Forest" ou "Gradient Boosting Decision Trees".
--n-estimators --max-depth	Tamanho do modelo. Em Uno/Nano, mantenha a profundidade baixa e poucas árvores.
--board / --fqbn	uno, nano, nano_old, mega, leonardo, esp32 — ou um FQBN explícito.
--port / --baud	Porta serial e velocidade (padrão 115200). Sem --port, tenta auto-detectar.
--n-samples / --reps	Quantas amostras enviar e quantas repetições por amostra na medição de latência.
--host-check	Liga a etapa 3 (compila o motor com g++ e compara com o Python).
--no-hardware	Para depois da etapa 3: não compila nem grava.
--skip-compile --skip-upload	Pula a compilação e/ou a gravação quando a placa já está pronta.
--min-parity	Paridade mínima para o teste passar (padrão 100%).
--output-dir / --report-dir	Onde ficam os artefatos gerados e os relatórios.

5. O protocolo serial

Texto puro, uma mensagem por linha, terminada em \n. Você pode conversar com a placa na mão pelo Monitor Serial da IDE — útil para depurar:

Sentido	Mensagem	Significado
PC -> placa	I	Pergunta os metadados do modelo.
placa -> PC	I,<algoritmo>,<n_features>,<n_classes>,<n_trees>	Resposta do handshake.
PC -> placa	P,<id>,<reps>,<f0>,<f1>,...	Pede a classificação de um vetor, repetida <reps> vezes.
placa -> PC	R,<id>,<classe>,<microsegundos>,<reps>	Classe prevista e tempo total das repetições.
placa -> PC	E,<motivo>	Erro de parsing ou comando desconhecido.

Exemplo manual, com um modelo de 4 features: enviar P, 0, 1, 5.1, 3.5, 1.4, 0.2 deve devolver algo como R, 0, 0, 180, 1 — classe 0, em 180 microssegundos.

6. Lendo o relatório

O .txt gerado tem esta cara:

```
HIL REPORT - pyruleanalyzer on hardware
Model       : Decision Tree (Decision Tree)
Board / FQBN : uno / arduino:avr:uno
Samples     : 50 (answered: 50)
```

```

C/Python parity    : 100.00%
Accuracy (host)    : 93.33%
Accuracy (board)   : 93.33%
Host-check (g++)   : 100.00% C/Python parity

Latency per inference: mean 212.4 us | p50 208.0 | p95 244.0 | max 260.0
Measured memory: Flash 3412 B (10%) | SRAM 268 B (13%)

RESULT: PASS

```

Campo	Como interpretar
C/Python parity	O número que importa: mede se a placa concorda com o Python amostra a amostra. Abaixo de 100%, o relatório lista as divergências (host, placa e rótulo verdadeiro).
Accuracy (host) × (board)	Acurácia contra o rótulo verdadeiro. Se as duas batem, o embarcado não perdeu qualidade.
Host-check (g++)	Paridade do mesmo C compilado no PC. Se falha aqui, o problema não é a placa.
Latency	Tempo por inferência (média, p50, p95 e máximo), já dividido pelas repetições. Use para dimensionar o período de amostragem.
Measured memory	Flash e SRAM reais reportados pelo toolchain, ao lado da estimativa do pyruleanalyzer.
RESULT	PASS = exit code 0; FAIL = exit code 1.

7. Armadilhas que você vai encontrar

- **No AVR, double é igual a float** (4 bytes). Os limiares são exportados com 17 dígitos e perdem precisão ao serem lidos pelo `atof()` da placa. Amostras que caem quase em cima de um limiar podem divergir do Python. Paridade de 99,x% com todas as divergências na fronteira é isso — e não um erro de lógica. No ESP32 (double de 8 bytes) o efeito some.
- **A ordem das features é um contrato.** A placa não conhece nomes, só índices. O vetor enviado precisa seguir a mesma ordem do treino — e `read_features()`, no sketch de produção, tem a mesma obrigação.
- **Uno e Nano têm 2 KB de SRAM.** Cada feature custa 4 bytes no buffer, mais cerca de 14 bytes de texto na linha serial. Acima de ~40 features o Uno aperta; prefira Mega ou ESP32.
- **Placas AVR resetam ao abrir a porta serial.** O script espera `--boot-delay` segundos (padrão 2,5 s) e descarta o banner antes do handshake.
- **ESP32 e o watchdog.** Muitas repetições numa única mensagem podem disparar o cão de guarda; mantenha `--reps` em até ~200.
- **Timeout.** Modelos grandes (Random Forest com dezenas de árvores) levam milissegundos por inferência em um Uno; multiplique isso por `--reps` antes de culpar o `--timeout`.

8. Problemas comuns

Sintoma	Causa provável e solução
<i>The board did not answer the "I" handshake</i>	Porta errada, baud diferente, sketch HIL não gravado, ou o Monitor Serial da IDE segurando a porta. Confira com <code>arduino-cli board list</code> e feche a IDE.

Sintoma	Causa provável e solução
arduino-cli not found	Instale o arduino-cli ou passe <code>--arduino-cli</code> C:\caminho\arduino-cli.exe. Para apenas gerar o sketch, use <code>--no-hardware</code> .
pyserial is not installed	<code>pip install pyserial</code> (o pacote se chama pyserial, mas o import é serial).
Compilação estourou a memória	Reduza <code>--max-depth</code> e <code>--n-estimators</code> , ou troque de placa. A estimativa já aparece na etapa 1.
Paridade abaixo de 100%	Rode com <code>--host-check</code> : se o g++ também divergir, o problema é o modelo/exportador; se só a placa divergir, quase sempre é a precisão de float no AVR.
Muitos timeouts	Aumente <code>--timeout</code> ou diminua <code>--reps</code> ; modelos grandes em AVR são lentos.

9. Usando em integração contínua

Sem placa, rode apenas a etapa 3 — ela garante que o exportador continua gerando C válido e equivalente ao Python:

```
- name: HIL host-check
  run: |
    pip install -e .
    python examples/arduino_hardware_test.py --dataset iris \
      --host-check --no-hardware --min-parity 100
```

Em um runner auto-hospedado com a placa física conectada, use o comando completo com `--port` fixo. Nos dois casos o exit code já funciona como critério de aprovação: não é preciso interpretar a saída.

10. Próximo passo: dos vetores do PC para os sensores reais

Depois que o teste HIL passa, você sabe que o motor de inferência está correto na placa. O que resta é trocar a fonte dos dados: no sketch de **produção**, preencha `read_features()` com as leituras dos seus sensores, **na mesma ordem de features usada no treino**:

```
void read_features(void) {
    features[0] = analogRead(A0) * (5.0 / 1023.0); // ex.: tensão do sensor 1
    features[1] = dht.readTemperature();           // ex.: temperatura
    features[2] = dht.readHumidity();               // ex.: umidade
    // ... uma linha por feature, na ordem do treino
}
```

Uma boa prática é manter os dois sketches à mão: o de produção para o campo e o HIL para revalidar o modelo sempre que ele for retreinado.